

Flink SQL reports

Five reports that read the isotope-tagged stream and surface what's flowing where, how fast, and how reliably. All 5 run as long-lived `INSERT INTO <report>_1m SELECT ...` streaming jobs on the cp-flink session cluster (Apache Flink 2.1.2). Sink topics use Apache Flink's `avro-confluent` SQL format — SR-framed Avro, auto-registered on first write — **Control Center deserializes them natively**.

The aggregation SQL is identical between Confluent Cloud for Apache Flink (CCAF) and Confluent Platform Flink (CP Flink on Minikube) — only the source-table and sink-table DDL differs (CCAF auto-manages Kafka tables via its topic catalog; CP Flink needs explicit `CREATE TABLE ... WITH ('connector' = 'kafka', ...)`).

Why Avro+SR (and not Protobuf+SR)?

The downstream demo *event* topics (`iso_start`, `iso_mid`, `iso_final`) ride Protobuf+SR via the Java app's `DemoEvent` schema. You might expect the report sinks to use the same format for consistency. They don't, and the reason is genuinely external:

- **cp-flink doesn't ship an SR-integrated Protobuf format.** Apache Flink open-source publishes `flink-sql-avro-confluent-registry` but no SR-integrated Protobuf equivalent.
- **CMF (Confluent Manager for Apache Flink) supports SR-Protobuf** via its `proto-registry` format, but **CMF Statements disallow user-defined functions**. The percentiles report's `LATENCY_PERCENTILES` T-Digest UDAF means we can't put that one in CMF, so a CMF migration would force a hybrid deployment (CMF for 4 reports + session cluster for the UDAF report). We tried that path earlier in this PR; rolled back in favor of one Flink version, one deployment model.
- **A hand-written SR-Protobuf format wrapper** is feasible (~150 lines wrapping `flink-protobuf` with magic-byte framing + SR client) but creates permanent maintenance surface for a problem Apache Flink already solves with Avro.

So: events from the app are Protobuf+SR (DemoEvent), aggregates from Flink are Avro+SR. Format-by-domain, not a defect.

Layout

```
flink/sql/
  cp/
    00_source_table.fql          # CREATE TABLE with 'connector' =
    'kafka' (reads iso-*)
    01_register_functions.fql    # Phase 2 – CREATE FUNCTION ... USING
JAR
    05_report_sinks.fql          # CREATE TABLE for each isotope-
report-*_1m Kafka sink (avro-confluent)
    99_teardown.fql             # DROP TABLE/VIEW/FUNCTION (companion
to flink-reports-down)
  cc/
    00_source_table.fql          # CREATE VIEW over CCAF's auto-
registered topic
```

```

    01_register_functions.fql      # Phase 2 – confluent-artifact:// JAR
reference
  shared/
    05_isotope_view.fql          # typed view; decodes x-isotope-*
header scalars
    10_latency_report.fql        # INSERT INTO: avg/min/max latency by
origin × topic
    20_topology_report.fql       # INSERT INTO: produce-edge counts
per minute
    30_hop_distribution.fql      # INSERT INTO: hop-count buckets per
topic per minute
    40_coverage_report.fql       # INSERT INTO: distinct traces per
topic per minute
    60_stuck_trace_report.fql     # Phase 2 – INSERT INTO: stuck-trace
alerts via STUCK_TRACE_PTF
    70_latency_percentiles_report.fql # Phase 2 – INSERT INTO: p50/p95/p99
via LATENCY_PERCENTILES UDAF

```

Wire-format detail

Window columns ride as `BIGINT` epoch millis on the wire, not `TIMESTAMP_LTZ`. Flink 2.1.2's `avro-confluent` schema-derivation path raises `UnsupportedOperationException: Unsupported to derive Schema for type: TIMESTAMP_LTZ(3)` for that type. We cast in the INSERT `(UNIX_TIMESTAMP(CAST(window_start AS STRING)) * 1000)` so the on-wire schema is plain Avro `long`. Consumers rehydrate via `TO_TIMESTAMP_LTZ(window_start, 3)`.

Operations

```

make flink-up          # cert-manager → CFK Flink Operator → CMF
(installed but unused for reports) → session cluster
make flink-reports-up  # build PTF JAR, copy to JM, create sink topics,
submit 6 INSERT INTO streaming jobs
make flink-sql         # interactive SQL Client (auto-loads sink DDL so
SELECT * works)
make flink-reports-down # cancel jobs, drop tables, delete sink topics + SR
subjects

```